

Workload Planning

Laboratory for the Information Systems course
in the degree program
Software Engineering and Media Computing

Milestone 4: Testing

submitted by

Team 4

Pavlos Arampatzis

Daniel Ryssel

Arbresha Selimi

Raluca Maria Vedislav

on December 3rd 2024

at Esslingen University of Applied Sciences

Table of Contents

Table of Contents	2
1 Test manual.....	3
1.1 Integrity Assurance.....	3
1.2 Business rules	3
1.3 Test Data.....	4
1.4 Test Cases	7

1 Test manual

The purpose of this test manual is to validate the integrity and correctness of the automatically generated physical database model, which is based on the logical database model created in the previous milestone. This validation ensures that all business rules are considered and implemented.

The manual outlines the complete testing process, including integrity assurance, a detailed list of business rules and their implementation, test data, and test cases.

1.1 Integrity Assurance

Integrity Assurance in the physical data model is achieved through several mechanisms that ensure the consistency and accuracy of the data stored in the database. These mechanisms include normalization up until the third form and the use of primary keys, foreign keys, data types, and constraints.

Primary Keys:

Each table in the model has a primary key to uniquely identify each entry. For example, the `COURSE` table uses a composite primary key consisting of `SUBJECT`, `TEACHER` and `COURSE_START`, ensuring that each course entry is distinct. Other tables like `FACULTY` and `TEACHER` have primary keys based on columns. These primary keys prevent duplicate entries in the tables.

Foreign Keys:

Foreign keys are used to establish relationships between tables and to ensure referential integrity. For example, the `COURSE` table contains foreign keys linking to the `SUBJECT` and `TEACHER` tables. These foreign keys ensure that any course entry has a valid subject and teacher in the referenced table, maintaining data consistency. It also allows to manage the multiplicity defined in the logical data model.

Data Types:

The model specifies specific data types for each column, such as `VARCHAR`, `INTEGER`, and `DECIMAL`. These data types ensure that the values stored in each column match the expected format.

Mandatory Constraint:

Several columns are marked as mandatory, meaning that they must always contain a value and cannot be null. For example, the `COURSE_START`, `SUBJECT`, and `TEACHER` col-

umns in the `COURSE` table are mandatory. These constraints prevent the creation of incomplete entries, ensuring that important information is always provided.

Attribute Domain Constraints:

The needed columns have additional constraints to ensure that attributes can only have values inside the specified domain. For example, the `CREDITED_TEACHER_HOURS` has a constraint that specifies that the value must be positive and below 10.

1.2 Business rules

The following business rules were thought of to ensure the correct functionality and integrity of the database. Each rule will be explained in more detail below, including the specific implementation.

1. Business Rule:

A lecturer cannot be a professor and lecturer at the same time. However, it is very possible for a lecturer to start as an assistant lecturer and be appointed professor in a subsequent semester. On the other hand, a lecturer who is a professor can work as a lecturer for a few semesters after retiring.

Implementation:

There exists a Boolean attribute called `IS_ACTIVE` in both the `PROFESSOR` and the `LECTURER` table to check if the teacher is currently active under that role.

If you want to change the role, you have to set the activity under the old role as inactive and create a new active entry in the other role. A trigger also needs to exist to check that a `TEACHER_ID` cannot exist in both tables simultaneously.

2. Business Rule:

There is no functional dependency between the hours that a lecture comprises according to the study and examination regulations and the hours that must be taken for this lecture in the timetable and the hours that a professor receives credit for as teaching load.

Implementation:

There are different columns for each of these hours, such as `ACTUAL_STUPO_HOURS`, `ACTUAL_SCHEDULE_HOURS` and `CREDITED_TEACHER_HOURS`. Therefore, no functional dependency exists.

3. Business Rule:

A subject can be taught by teachers from different faculties in the same semester

Implementation:

The primary key of the table `COURSE` consists of the attributes `COURSE_START`, `SUBJECT` and `TEACHER`. Due to this, a subject can be taught by multiple teachers in the same semester. The faculty belongs to the teacher, therefore there is no constraint here.

4. Business Rule:

A lecturer is always assigned to one faculty.

Implementation:

The `TEACHER` table has a foreign key called `FACULTY` and specifies the faculty they belong to. The attribute is mandatory, so a lecturer always belongs to one faculty.

5. Business Rule:

Subjects must be correctly assigned to the appropriate semester level and degree program.

Implementation:

The `SUBJECT` table has attributes such as the `SEMESTER` and `MAJOR`, which specify the appropriate semester level and degree program.

6. Business Rule:

A retired professor cannot teach after retirement unless they are assigned as an adjunct professor.

Implementation:

There exists a Boolean attribute called `IS_ACTIVE` in the `PROFESSOR` table to check if the teacher is currently active under that role.

If a professor retires, the `IS_ACTIVE` attribute needs to be changed to false.

Then, if they want to continue working as an adjunct professor a new teacher entry can be created and inserted in the lecturer table.

7. Business Rule:

A lecturer cannot teach more hours than the defined workload capacity (18 hours per week). For a professor, the workload above the 18 compulsory hours is credited to a credit hour account.

Implementation:

A new table called `SEARCH_CRITERIA` should be created, which contains the `MAJOR`, `SEMESTER` and `SEMESTER_START`. When the `SEMESTER_START` gets changed, the workload and credit hour account get automatically calculated. This will be done with the help of a trigger and a stored procedure.

8. Business Rule:

Credited hours are carried over per semester and billed from, when the weekly hours are lower than 18 compulsory hours.

Implementation:

The carryover is done automatically because the `CREDIT_HOUR_ACCOUNT` is not bound to a semester (such as WS24/25). The billing is automatically done when the `SEMESTER_START` of the `SEARCH_CRITERIA` gets changed.

9. Business Rule:

A service doesn't have an assigned semester level and degree program.

Implementation:

The `SUBJECT` table has attributes such as the `SEMESTER` and `MAJOR`, but they are not mandatory. In case of a service, these attributes can be left empty.

10. Business Rule:

A laboratory can be held multiple times, if the amount of people attending is too much for one appointment.

Implementation:

The user that enters the credited hours already has the freedom to enter double the amount if the laboratory is held twice.

11. Business Rule:

One lecture can be held by multiple professors. Doubling of credited hours should be avoided.

Implementation:

The user that enters the credited hours already has the freedom to enter half the amount if the lecture is held by two different teachers. The user also has the freedom to choose how to split the credited hours.

12. Business Rule:

Lecturers can't have a deputyship.

Implementation:

Lecturers have their own table, which is not referenced by the `FUNCTIONS` table. Therefore it is impossible for them to have a function.

13. Business Rule:

Timetable hours (`ACTUAL_SCHEDULE_HOURS`) are divisible by 2.

Implementation:

Create a constraint which checks if the input is divisible by 2.

14. Business Rule:

The load input of a function has to be positive and lower than 10.

Implementation:

Create a constraint which checks if the input is between 0 and 10.

15. Business Rule:

The hour entries need to be plausible.

Implementation:

Create a constraint which checks if the input is between 0 and 10. 0.

16. Business Rule:

A semester needs to be between 1 and 7.

Implementation:

Create a constraint which checks if the input is between, and including, 1 and 7.

1.3 Test Data

The test data is a subset of real data used to verify the correctness and validity of the system, ensuring that all business rules are properly implemented. This data serves as a foundation to thoroughly test both valid and invalid scenarios that may occur.

The test data is comprehensive enough to cover the following cases:

- Coverage of at least two consecutive semesters
- Professors and lecturers
- Coupling of lectures
- Duplication of lectures by laboratory groups
- Professors in self-administration functions
- Credit carryover from previous semesters
- Billed semesters, current semesters, and semesters in planning
- Lecturers with time accounts in other faculties
- Service from other faculties
- Service for other faculties

The extraction of data was performed using a Python script, which formatted the data accordingly. The test data is attached at the end of this document.

1.4 Test Cases

These cases are derived from the business rules and the data described in the test data section. Each test case will validate specific functionality, consistency, and integrity of the system.

Before testing, the following steps were completed as a preparation:

- Creating the tables in the project database based on the physical data model
- Entering the test data

Test Case 1: Teacher and Professor Simultaneously

- Description: It is not possible for a person to be a professor and a lecturer at the same time. However, it is possible for a lecturer to advance to a professor, and it is possible for a professor to work as a lecturer after retiring.
- Preconditions: A professor exists and the attribute `IS_ACTIVE` is true.

- Test Steps:
 1. Create a lecturer whose `TEACHER_ID` exists in the `PROFESSOR` table.
 2. Try to put the attribute `IS_ACTIVE` on true in the `LECTURER` table for the same teacher who is a professor.
- Expected Result: The system doesn't allow the entry in the `LECTURER` table.
- Actual Result: The system allows both steps to happen.

Test Case 2: Functional Dependency Between Hours

- Description: There is no functional dependency between the hours that a lecturer comprises according to the study and examination regulations and the hours that must be taken for this lecture in the timetable and the hours that a professor receives credit for as teaching load.
- Preconditions: None
- Test Steps:
 1. Try to insert, update or delete a data set in the `COURSES` table with independent hours.
- Expected Result: The system allows inserting, independently updating or deleting a data set containing `ACTUAL_STUPO_HOURS`, `ACTUAL_SCHEDULE_HOURS` and `CREDITED_TEACHER_HOURS`.
- Actual Result: The actual result corresponds to the expected result.

Test Case 3: Splitting a Course Between Teachers

- Description: A subject can be taught by lecturers from different faculties in the same semester.
- Preconditions: A course existing.
- Test Steps:
 1. Create a new course with the same `COURSE_START` and `SUBJECT` as the course already existing but with a different teacher.
- Expected Result: The system allows the creation of a new course with the same `COURSE_START` and `SUBJECT` and with a different teacher. The faculty can be different.
- Actual Result: The actual result corresponds to the expected result.

Test Case 4: Lecturer Faculty

- Description: A lecturer is always assigned to one faculty.
- Preconditions: None
- Test Steps:
 1. Create an entry in the `TEACHER` table but leave the faculty empty.
- Expected Result: The system doesn't allow creating a teacher without a faculty.
- Actual Result: The actual result corresponds to the expected result.

Test Case 5: Subject Semester and Degree Program

- Description: Subjects must be correctly assigned to the appropriate semester and degree program. Subjects have a semester and major attribute.
- Preconditions: None
- Test Steps:
 1. Create a subject with an appropriate semester and major.
 2. Create a course with that subject.
- Expected Result: The system allows this. Courses can only be held for the correct semester.
- Actual Result: The actual result corresponds to the expected result.

Test Case 6: Retired Professors

- Description: A retired professor cannot teach after retirement unless they are assigned as an adjunct professor. Professors have the `IS_ACTIVE` attribute that needs to be true for them to be assigned to a subject.
- Preconditions: A professor exists and the attribute `IS_ACTIVE` is false.
- Test Steps:
 1. Try to create a course with a teacher, who is a professor with the `IS_ACTIVE` attribute on false.
- Expected Result: The insertion fails.
- Actual Result: The trigger is not implemented – the insertion succeeds.

Test Case 7.1: Lecturer Maximum Workload (18 Hours)

- Description: A lecturer cannot teach more than 18 hours. When a lecturer is assigned a new course, their total hours per week for that semester are calculated and if they exceed 18 hours, the insertion should fail.
- Preconditions: A lecturer exists and the attribute `IS_ACTIVE` is true.
- Test Steps:
 1. Create a few courses and assign the lecturer. The workload should be less than 18 hours.
 2. Create another course for the same semester and assign the lecturer. The workload of 18 hours should be exceeded now.
- Expected Result: The system doesn't allow the second assignment.
- Actual Result: The trigger is not implemented – the assignment succeeds.

Test Case 7.2: Professor Credited Hours

- Description: A professor is credited for overtime. Professors have a the `CREDIT_HOUR_ACCOUNT` attribute where the surplus/deficit of hours per week is stored.
- Preconditions: A professor exists and the attribute `IS_ACTIVE` is true.
- Test Steps:
 1. Assign the professor to multiple courses. They should account for more than 18 hours.
 2. Check the credited hours.
- Expected Result: The `CREDIT_HOUR_ACCOUNT` contains the workload minus 18.
- Actual Result: The credit hour account does not get updated at all, the user would have to update it themselves.

Test Case 8: Credit Hour Account Billing

- Description: Credited hours are carried over per semester and billed from.
- Preconditions: A professor exists and the attribute `IS_ACTIVE` is true. The professor is assigned to courses and functions from at least 2 different semesters and has 10 credited hours.

- Test Steps:
 1. Create a new semester and assign no courses to the professor.
 2. Check the credit hour account
- Expected Result: The CREDIT_HOUR_ACCOUNT should be negative.
- Actual Result: The credit hour account does not get updated at all, the user would have to update it themselves.

Test Case 9: Major and Semester Level of Services

- Description: A service doesn't need to have an assigned semester level and degree program.
- Preconditions: None
- Test Steps:
 1. Create a new subject that is meant to be a service. Don't fill out the fields, where a service doesn't have data (semester level and major).
- Expected Result: The system allows the creation of a subject (service) without a semester level and major.
- Actual Result: The actual result corresponds to the expected result.

Test Case 10: Holding a Laboratory Multiple Times

- Description: A laboratory can be held more than one time in a week, if the amount of people attending is too much for one appointment.
- Preconditions: A laboratory with an assigned teacher already exists in the courses table.
- Test Steps:
 1. Change the credited hours to double the amount, if the laboratory is held two times.
- Expected Result: The system allows for a laboratory to be "split". This means that the credited hours are simply doubled or multiplied by the times that the laboratory is held.
- Actual Result: This works only for enough "splits" until the upper constraint of 10 is reached for the credited teacher hours.

Test Case 11: Workload when Splitting a Course Between Teachers

- Description: One lecture can be held by multiple professors. Doubling of credited hours should be avoided.
- Preconditions: A course with an assigned teacher already exists in the courses table.
- Test Steps:
 1. Add another course with the same subject and another teacher.
 2. (Change the credited hours for the split course)
- Expected Result: The system allows for courses to be split between multiple teachers. The number of credited hours shouldn't be doubled.
- Actual Result: The course can be split between multiple teachers, but you are not forced to change the credited hours. The responsibility lies in the user.

Test Case 12: Lecturer Deputyship

- Description: Lecturers can't have a deputyship.
- Preconditions: A lecturer exists in the system.
- Test Steps:
 1. Attempt to create a function. Insert a lecturer ID in the Professor column.
- Expected Result: It is not possible to create a function with an ID of a lecturer and not professor.
- Actual Result: The actual result corresponds to the expected result.

Test Case 13: Timetable hours divisible by 2

- Description: Timetable hours (ACTUAL_SCHEDULE_HOURS) are divisible by 2.
- Precondition: None
- Test Steps:
 1. Insert a course with an uneven number of hours.
- Expected Result: The system doesn't allow the insertion of the new course.
- Actual Result: The actual result corresponds to the expected result.

Test Case 14: Load Input of a Function

- Description: The load input of a function has to be positive and lower than 10.
- Preconditions: None

- Test Steps:
 1. Attempt to create a function. Insert a load input for the function of -1.
 2. Attempt to create a function. Insert a load input for the function of 4.
 3. Attempt to create a function. Insert a load input for the function of 12.
- Expected Result: First and third test step fail and get rejected, second function can get inserted.
- Actual Result: The actual result corresponds to the expected result.

Test Case 15: Plausible hours

- Description: The hour entries need to be plausible.
- Preconditions: None
- Test Steps:
 1. Create a new course with 15 actual schedule hours per week.
- Expected Result: The course can't get created.
- Actual Result: The actual result corresponds to the expected result.

Test Case 16: Valid Semester

- Description: The semester value needs to be between 1 and 7.
- Preconditions: None
- Test Steps:
 1. Create a new subject and select 8 as the semester value.
- Expected Result: It is not possible to create this subject.
- Actual Result: The actual result corresponds to the expected result.